JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class SynchronousQueue<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractQueue<E>
      java.util.concurrent.SynchronousQueue<E>

Type Parameters:

`E - the type of elements held in this collection`

All Implemented Interfaces:

`Serializable, Iterable<E>, Collection<E>, BlockingQueue<E>, Queue<E>`

```
public class SynchronousQueue<E>
extends AbstractQueue<E>
implements BlockingQueue<E>, Serializable
```

A blocking queue in which each insert operation must wait for a corresponding remove operation by another thread, and vice versa. A synchronous queue does not have any internal capacity, not even a capacity of one. You cannot `peek` at a synchronous queue because an element is only present when you try to remove it; you cannot insert an element (using any method) unless another thread is trying to remove it; you cannot iterate as there is nothing to iterate. The *head* of the queue is the element that the first queued inserting thread is trying to add to the queue; if there is no such queued thread then no element is available for removal and `poll()` will return `null`. For purposes of other `Collection` methods (for example `contains`), a `SynchronousQueue` acts as an empty collection. This queue does not permit `null` elements.

Synchronous queues are similar to rendezvous channels used in CSP and Ada. They are well suited for handoff designs, in which an object running in one thread must sync up with an object running in another thread in order to hand it some information, event, or task.

This class supports an optional fairness policy for ordering waiting producer and consumer threads. By default, this ordering is not guaranteed. However, a queue constructed with fairness set to `true` grants threads access in FIFO order.

This class and its iterator implement all of the *optional* methods of the `Collection` and `Iterator` interfaces.

This class is a member of the Java Collections Framework.

Since:

`1.5`

See Also:

`Serialized Form`

Constructor Summary

Constructors

Constructor	Description
`SynchronousQueue()`	Creates a `SynchronousQueue` with nonfair access policy.
`SynchronousQueue(boolean fair)`	Creates a `SynchronousQueue` with the specified fairness policy.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`void`	`clear()`	Does nothing.
`boolean`	`contains(Object o)`	Always returns `false`.
`boolean`	`containsAll(Collection<?> c)`	Returns `false` unless the given collection is empty.
`int`	`drainTo(Collection<? super E> c)`	Removes all available elements from this queue and adds them to the given collection.
`int`	`drainTo(Collection<? super E> c, int maxElements)`	Removes at most the given number of available elements from this queue

		and adds them to the given collection.
boolean	**isEmpty**()	Always returns true.
Iterator<**E**>	**iterator**()	Returns an empty iterator in which hasNext always returns false.
boolean	**offer**(**E** e)	Inserts the specified element into this queue, if another thread is waiting to receive it.
boolean	**offer**(**E** e, long timeout, **TimeUnit** unit)	Inserts the specified element into this queue, waiting if necessary up to the specified wait time for another thread to receive it.
E	**peek**()	Always returns null.
E	**poll**()	Retrieves and removes the head of this queue, if another thread is currently making an element available.
E	**poll**(long timeout, **TimeUnit** unit)	Retrieves and removes the head of this queue, waiting if necessary up to the specified wait time, for another thread to insert it.
void	**put**(**E** e)	Adds the specified element to this queue, waiting if necessary for another thread to receive it.
int	**remainingCapacity**()	Always returns zero.
boolean	**remove**(**Object** o)	Always returns false.
boolean	**removeAll**(**Collection**<?> c)	Always returns false.
boolean		Always returns false.

	retainAll(**Collection**<? > c)	
int	**size**()	Always returns zero.
Spliterator<**E**>	**spliterator**()	Returns an empty spliterator in which calls to **Spliterator.trySplit()** always return `null`.
E	**take**()	Retrieves and removes the head of this queue, waiting if necessary for another thread to insert it.
Object[]	**toArray**()	Returns a zero-length array.
<T> T[]	**toArray**(T[] a)	Sets the zeroeth element of the specified array to `null` (if the array has non-zero length) and returns it.

Methods inherited from class java.util.AbstractQueue

add, addAll, element, remove

Methods inherited from class java.util.AbstractCollection

toString

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.concurrent.BlockingQueue

add

Methods inherited from interface java.util.Queue

element, remove

Methods inherited from interface java.util.Collection

addAll, equals, hashCode, parallelStream, removeIf, stream

Methods inherited from interface java.lang.Iterable

forEach

Constructor Detail

SynchronousQueue

```
public SynchronousQueue()
```

Creates a SynchronousQueue with nonfair access policy.

SynchronousQueue

```
public SynchronousQueue(boolean fair)
```

Creates a SynchronousQueue with the specified fairness policy.

Parameters:

fair - if true, waiting threads contend in FIFO order for access; otherwise the order is unspecified.

Method Detail

put

```
public void put(E e)
         throws InterruptedException
```

Adds the specified element to this queue, waiting if necessary for another thread to receive it.

Specified by:

put in interface BlockingQueue<E>

Parameters:

e - the element to add

Throws:

InterruptedException - if interrupted while waiting

NullPointerException - if the specified element is null

offer

```
public boolean offer(E e,
                     long timeout,
                     TimeUnit unit)
              throws InterruptedException
```

Inserts the specified element into this queue, waiting if necessary up to the specified wait time for another thread to receive it.

Specified by:

offer in interface BlockingQueue<E>

Parameters:

e - the element to add

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

Returns:

true if successful, or false if the specified waiting time elapses before a consumer appears

Throws:

InterruptedException - if interrupted while waiting

NullPointerException - if the specified element is null

offer

```
public boolean offer(E e)
```

Inserts the specified element into this queue, if another thread is waiting to receive it.

Specified by:

offer in interface BlockingQueue<E>

Specified by:

offer in interface Queue<E>

Parameters:

e - the element to add

Returns:

true if the element was added to this queue, else false

Throws:

NullPointerException - if the specified element is null

take

```
public E take()
       throws InterruptedException
```

Retrieves and removes the head of this queue, waiting if necessary for another thread to insert it.

Specified by:

take in interface BlockingQueue<E>

Returns:

the head of this queue

Throws:

InterruptedException - if interrupted while waiting

poll

```
public E poll(long timeout,
              TimeUnit unit)
       throws InterruptedException
```

Retrieves and removes the head of this queue, waiting if necessary up to the specified wait time, for another thread to insert it.

Specified by:

poll in interface BlockingQueue<E>

Parameters:

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

Returns:

the head of this queue, or null if the specified waiting time elapses before an element is present

Throws:

InterruptedException - if interrupted while waiting

poll

```
public E poll()
```

Retrieves and removes the head of this queue, if another thread is currently making an element available.

Specified by:

`poll` in interface `Queue<E>`

Returns:

the head of this queue, or null if no element is available

isEmpty

```
public boolean isEmpty()
```

Always returns `true`. A `SynchronousQueue` has no internal capacity.

Specified by:

`isEmpty` in interface `Collection<E>`

Overrides:

`isEmpty` in class `AbstractCollection<E>`

Returns:

`true`

size

```
public int size()
```

Always returns zero. A `SynchronousQueue` has no internal capacity.

Specified by:

`size` in interface `Collection<E>`

Specified by:

`size` in class `AbstractCollection<E>`

Returns:

zero

remainingCapacity

```
public int remainingCapacity()
```

Always returns zero. A `SynchronousQueue` has no internal capacity.

Specified by:

`remainingCapacity` in interface `BlockingQueue<E>`

Returns:

zero

clear

```
public void clear()
```

Does nothing. A SynchronousQueue has no internal capacity.

Specified by:

clear in interface Collection<E>

Overrides:

clear in class AbstractQueue<E>

contains

```
public boolean contains(Object o)
```

Always returns false. A SynchronousQueue has no internal capacity.

Specified by:

contains in interface Collection<E>

Specified by:

contains in interface BlockingQueue<E>

Overrides:

contains in class AbstractCollection<E>

Parameters:

o - the element

Returns:

false

remove

```
public boolean remove(Object o)
```

Always returns false. A SynchronousQueue has no internal capacity.

Specified by:

remove in interface Collection<E>

Specified by:

remove in interface BlockingQueue<E>

Overrides:

remove in class AbstractCollection<E>

Parameters:

`o - the element to remove`

Returns:

`false`

containsAll

```
public boolean containsAll(Collection<?> c)
```

Returns `false` unless the given collection is empty. A `SynchronousQueue` has no internal capacity.

Specified by:

`containsAll in interface Collection<E>`

Overrides:

`containsAll in class AbstractCollection<E>`

Parameters:

`c - the collection`

Returns:

`false unless given collection is empty`

See Also:

`AbstractCollection.contains(Object)`

removeAll

```
public boolean removeAll(Collection<?> c)
```

Always returns `false`. A `SynchronousQueue` has no internal capacity.

Specified by:

`removeAll in interface Collection<E>`

Overrides:

`removeAll in class AbstractCollection<E>`

Parameters:

`c - the collection`

Returns:

`false`

See Also:

`AbstractCollection.remove(Object)`,
`AbstractCollection.contains(Object)`

retainAll

```
public boolean retainAll(Collection<?> c)
```

Always returns `false`. A `SynchronousQueue` has no internal capacity.

Specified by:

`retainAll` in interface `Collection<E>`

Overrides:

`retainAll` in class `AbstractCollection<E>`

Parameters:

`c` - the collection

Returns:

`false`

See Also:

`AbstractCollection.remove(Object)`, `AbstractCollection.contains(Object)`

peek

```
public E peek()
```

Always returns `null`. A `SynchronousQueue` does not return elements unless actively waited on.

Specified by:

`peek` in interface `Queue<E>`

Returns:

`null`

iterator

```
public Iterator<E> iterator()
```

Returns an empty iterator in which `hasNext` always returns `false`.

Specified by:

`iterator` in interface `Iterable<E>`

Specified by:

`iterator` in interface `Collection<E>`

Specified by:

`iterator` in class `AbstractCollection<E>`

Returns:

an empty iterator

spliterator

```
public Spliterator<E> spliterator()
```

Returns an empty spliterator in which calls to `Spliterator.trySplit()` always return `null`.

Specified by:

`spliterator` in interface `Iterable<E>`

Specified by:

`spliterator` in interface `Collection<E>`

Returns:

an empty spliterator

Since:

1.8

toArray

```
public Object[] toArray()
```

Returns a zero-length array.

Specified by:

`toArray` in interface `Collection<E>`

Overrides:

`toArray` in class `AbstractCollection<E>`

Returns:

a zero-length array

toArray

```
public <T> T[] toArray(T[] a)
```

Sets the zeroeth element of the specified array to `null` (if the array has non-zero length) and returns it.

Specified by:

`toArray` in interface `Collection<E>`

Overrides:

`toArray` in class `AbstractCollection<E>`

Type Parameters:

T - the runtime type of the array to contain the collection

Parameters:

a - the array

Returns:

the specified array

Throws:

NullPointerException - if the specified array is null

drainTo

```
public int drainTo(Collection<? super E> c)
```

Description copied from interface: BlockingQueue

Removes all available elements from this queue and adds them to the given collection. This operation may be more efficient than repeatedly polling this queue. A failure encountered while attempting to add elements to collection c may result in elements being in neither, either or both collections when the associated exception is thrown. Attempts to drain a queue to itself result in IllegalArgumentException. Further, the behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Specified by:

drainTo in interface BlockingQueue<E>

Parameters:

c - the collection to transfer elements into

Returns:

the number of elements transferred

Throws:

UnsupportedOperationException - if addition of elements is not supported by the specified collection

ClassCastException - if the class of an element of this queue prevents it from being added to the specified collection

NullPointerException - if the specified collection is null

IllegalArgumentException - if the specified collection is this queue, or some property of an element of this queue prevents it from being added to the specified collection

drainTo

```
public int drainTo(Collection<? super E> c,
                   int maxElements)
```

Description copied from interface: BlockingQueue

Removes at most the given number of available elements from this queue and adds them to the given collection. A failure encountered while attempting to add elements to collection `c` may result in elements being in neither, either or both collections when the associated exception is thrown. Attempts to drain a queue to itself result in `IllegalArgumentException`. Further, the behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Specified by:

`drainTo` in interface `BlockingQueue<E>`

Parameters:

`c` - the collection to transfer elements into

`maxElements` - the maximum number of elements to transfer

Returns:

the number of elements transferred

Throws:

`UnsupportedOperationException` - if addition of elements is not supported by the specified collection

`ClassCastException` - if the class of an element of this queue prevents it from being added to the specified collection

`NullPointerException` - if the specified collection is null

`IllegalArgumentException` - if the specified collection is this queue, or some property of an element of this queue prevents it from being added to the specified collection

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.